UCD Michael Smurfit Graduate Business School

Name: Shruti Avinash Punjabi
Student ID : 25201664
MSc in Financial Data Science

Module: Data Science for Trading & Risk Management
FIN42110: Homework 2
Module Leader: Richard McGee
Due: February 27th 2026

TABLE OF CONTENTS

PART 1 - Web Scraping & Summary Statistics **3**
Summary Statistics 3
Trades per week 3
Top 5 Traders 3
PART 2 - ETH Data **4**
PART 3 - NFT Index & Hedonic Pricing Model **5**
Sample Restriction 5
Rarity Measures 5
Hedonic Regression Results 6
PART 4 - Trading Strategy **7**
Strategy Implementation 7
Performance Results 7
Limitations 7
PART 5 - Appendix **8**

PART 1 - Web Scraping & Summary Statistics

The assigned cryptopunks (IDs 7879-8181) were scraped and stored in SQLite database containing tables : Punks , PunkTrades , EthPrices , and PunkIndexCSM.

Summary Statistics

Trading Activity

- Average number of actual sales per punk : 3.98
- Median number of sales : 3
- Standard Deviation : 2.92
- 22.6% of punks traded only once, indicating moderate illiquidity.
- The top 10% of punks account for 25.1% of all trades, suggesting concentrated trading activity.

This distribution reflects that while some NFTs trade frequently, a large fraction exhibit lower turnover

Market Extremes

- Highest Sale : Punk 7996 sold $739,356
- Most frequently traded punk : Punk 8091 (149 trades)

This data confirms that there is strong heterogeneity in liquidity and valuation across NFTs.

Trades per week

Weekly trades ranged from low single digits in early years to peak activity during the 2021-2022, coinciding with broader NFT & crypto bull market , and declined post 2022.This indicates strong cyclical market dynamics.

Top 5 Traders

Trader	Number of Trades	Avg Net Position (USD)
0xb4ea05	572	205.15
punksotc…	228	65,183.95
0x717403	85	13.96
Pranksy	72	11,668.61
0xe186a8	47	135,411.06

Table 1- reflects the Top 5 Traders

Net position is computed as total sales - total purchases
This table suggests the presence of dominant liquidity providers or speculative accounts.

PART 2 - ETH Data

Daily ETH - USD prices were downloaded from FRED (18-05-2016 - latest available) website and stored in the ETHPrices database.

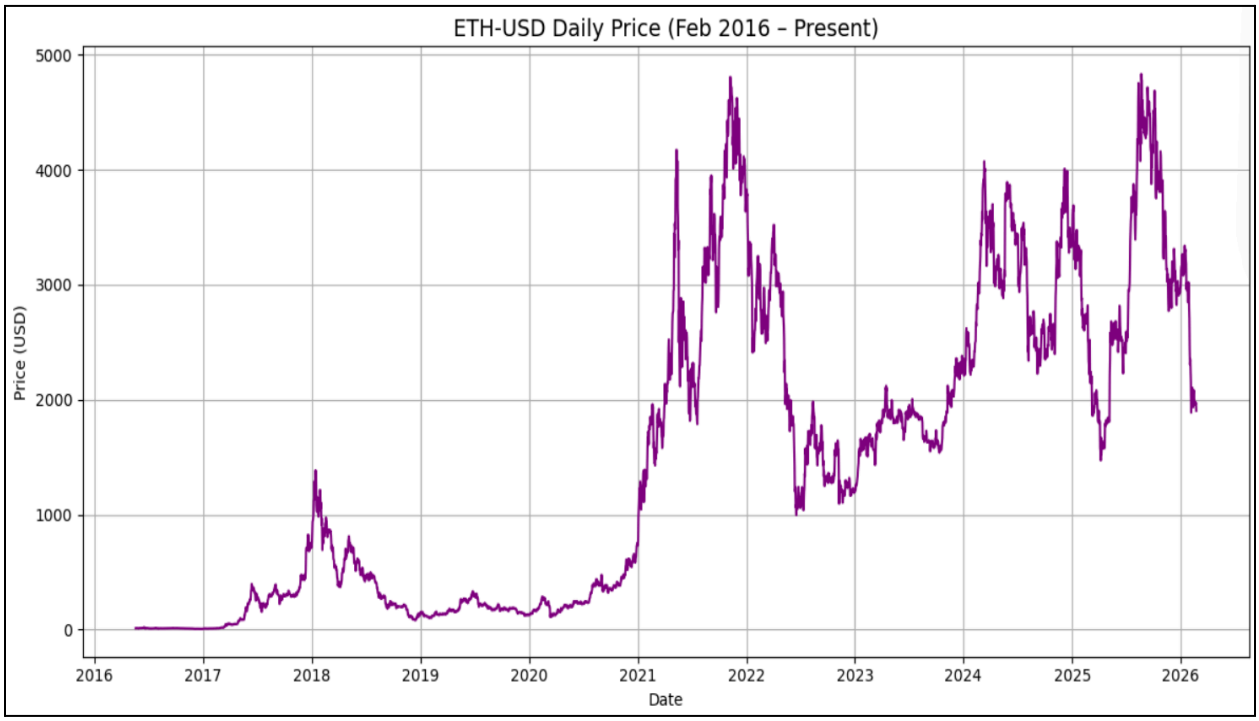

Figure 1 plots the Eth-USD Daily price from maximum availability (2016 to present)

Figure 1 concludes that the ETH series has significant volatility , with strong growth into 2021 followed by a correction phase.

PART 3 - NFT Index & Hedonic Pricing Model

Sample Restriction

Only NFTs with at least 2 sales were included (190 punks), allowing construction of a repeat - sales index.

Case-Shiller Repeat - Sales Index

A true repeat- sales regression was implemented using log price differences between consecutive sales of the same NFT.
The index was normalised to 100 in the base period.

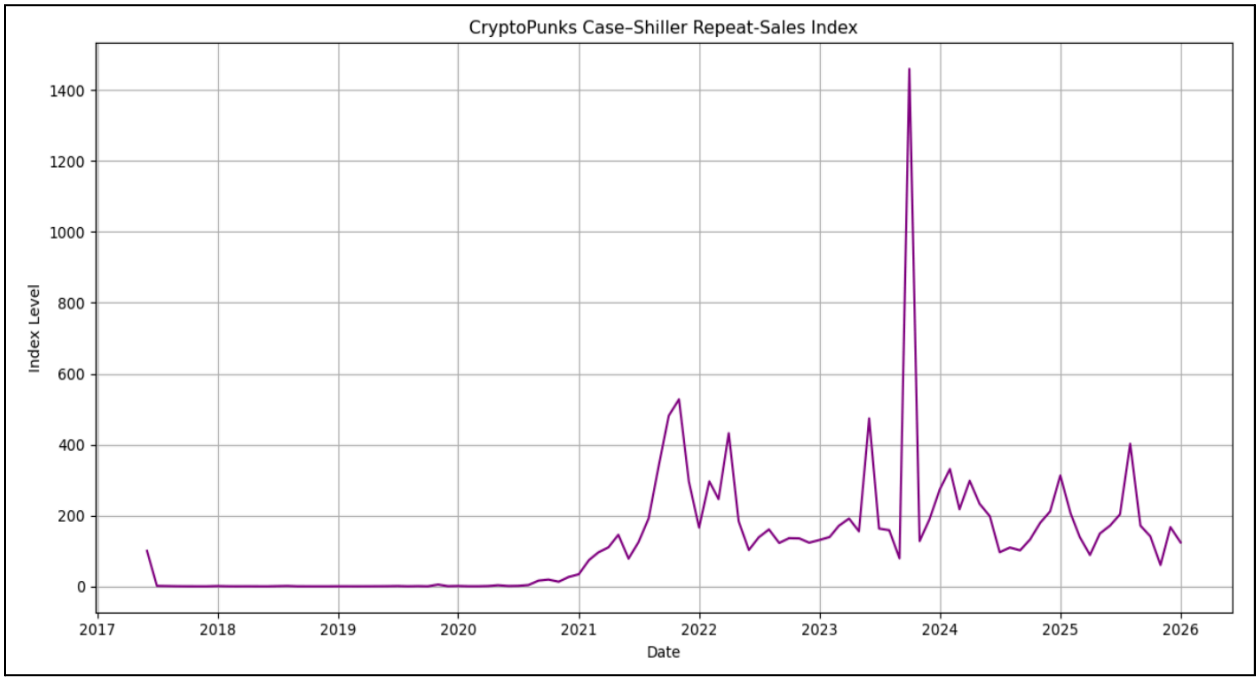

Figure 2 plot shows the true repeat - sales regression for yearly basis

Figure 2 indicates that early periods show limited movements due to sparse repeat sales prior to 2020. Index dynamics increase substantially as market liquidity expands from 2020 onwards. This plot reflects rising liquidity and speculative dynamics in the NFT market.

Rarity Measures

Three Rarity proxies were constructed:

1. Minimum - attribute rarity:
    - log(minimum attribute frequency)
2. Sum-based rarity score
   $\sum -\log(pk)$ across all traits
3. Number of traits (collectibility proxy)

The sum- based rarity measure captures the cumulative scarcity of all attributes and provides a measure of uniqueness.

Hedonic Regression Results

Model estimated:

$$\ln(P_i) = \beta_0 + \beta_1 Rarity_i + \beta_2 Traits_i + \beta_3 \ln(MarketIndex_t) + \beta_4 \ln(ETH_t) + \beta_5 Type_i + \varepsilon_i$$

Dependent Variable : LogPrice USD
In sample observations (⅔ rd data) : 42,420

Variable	**Coefficient**	**p-Value**
Constant	-5.8197	0.000
Rarity SumScore	0.0988	0.000
Num Traits	-0.1400	0.000
LogIndex	0.2861	0.000
LogETHUSD	2.0424	0.000
PunkType (Dummy 4)	-0.0487	0.000

Table 2 reflects the Regression output values

R^2 = 0.866
Adjusted R^2 = 0.866

Interpretation

- RaritySumScore has a positive and statistically significant effect on NFT prices ((β = 0.0.0988) indicating that greater cumulative attribute scarcity increases NFT Prices.
- The Case-shiller NFT index is strongly significant (β = 0.2861) , confirming that broader NFT market movements affect individual prices.
- ETH Price has a strong and highly significant impact on NFT prices (β = 2.144), implying NFTs exhibit amplified exposure to crypto market conditions.
- The model explains 86.6% of price variation in-sample.

PART 4 - Trading Strategy

Strategy Implementation

The model was evaluated over the first 6 months of the out-of-sample period.

- BUY if market price < model valuation.
- SHORT if market price > model valuation.
- Close positions at the next observed sale.
- Only one position per NFT was allowed at any time.

Performance Results

- Closed trades: 146
- Average return per trade: -2.09%
- The strategy produced short signals (139 SHORT vs. 22 BUY).

The negative average return indicates mispricing signals derived from the hedonic model lacked out-of-sample predictive power, indicating structural instability or model overfitting.

Limitations

1. **Illiquidity risk** : Although the average number of sales per punk is 3.98, 22.6% of NFTs traded only once, which imply limited exit opportunities and significant holding risk.
2. **Short Concentration**: The strategy generated 139 SHORT signals vs only 22 BUY signals , yet short selling is not feasible in practice, limited real-world implementation.
3. **Thin return margin :** The average return per trade was -2.09%, meaning realistic transaction costs (gas,fees, marketplace commissions, spreads) would likely further worsen performance.
4. **Model Instability :** Despite strong in-sample fit (Adj. R2 = 86.6%), the negative out-of-sample return suggests structural instability or regime shifts in NFTs pricing.
5. **Serial Correlation :** The DB statistics (0.159) indicates residual autocorrelation, potentially biasing inference and weakening predictive signals.
6. **Signal Sensitivity** : Applying 10%-20% mispricing threshold reduced trade frequency but it further worsened the out_of_sample returns, which strongly suggest that large valuation gaps are driven by model error rather than genuine arbitrage opportunities.

References

1. Ppts Uploaded on Brightspace.
2. Files Uploaded on Brightspace.
3. LLM assistance for Task 4.

PART 5 - Appendix

Task B: SQL Queries

```
import sqlite3
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import requests
from bs4 import BeautifulSoup  # for scraping
import time
import pandas as pd
import matplotlib.pyplot as plt
from pandas_datareader import data as pdr
import datetime as dt
import statsmodels.api as sm
from statsmodels.formula.api import ols

DB_NAME = "punkDB.db"
import sqlite3
import pandas as pd

DB_NAME = "punkDB.db"   # or "punkDBFull.db" if that’s what you used

conn = sqlite3.connect(DB_NAME)
conn.execute("PRAGMA journal_mode=WAL;")

# Confirm tables exist
pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    conn
)

# Average number of actual trades per punk
pd.read_sql_query("""
SELECT AVG(cnt) AS avg_trades_per_punk
FROM (
    SELECT PunkID, COUNT(*) AS cnt
    FROM PunkTrades
    WHERE TType='Sold'
    GROUP BY PunkID
);
""", conn)

# Median trades per punk
trade_counts = pd.read_sql_query("""
SELECT PunkID, COUNT(*) AS TradeCount
FROM PunkTrades
WHERE TType='Sold'
GROUP BY PunkID
""", conn)
trade_counts["TradeCount"].median()

# Trades per week
pd.read_sql_query("""
```

```
SELECT
    strftime('%Y-%W',
             substr(TDate,1,4)||'-'||substr(TDate,5,2)||'-'||substr(TDate,7,2)
    ) AS YearWeek,
    COUNT(*) AS TradesPerWeek
FROM PunkTrades
GROUP BY YearWeek
ORDER BY YearWeek;
""", conn)

#Standard Deviation of Trades per punk
print("Std deviation:", trade_counts["TradeCount"].std())

# Percentage of Punks that traded only once
one_trade_share = (trade_counts["TradeCount"] == 1).mean()
print("Share traded only once:", round(one_trade_share * 100, 2), "%")

trade_counts_sorted = trade_counts.sort_values("TradeCount", ascending=False)

top10_n = int(len(trade_counts_sorted) * 0.10)
top10_share = (
    trade_counts_sorted.head(top10_n)["TradeCount"].sum()
    / trade_counts_sorted["TradeCount"].sum()
)

print("Top 10% share:", round(top10_share * 100, 2), "%")

# Highest priced sale
pd.read_sql_query("""
SELECT PunkID, TAmt
FROM PunkTrades
WHERE TType='Sold' AND TAmt>0
ORDER BY TAmt DESC
LIMIT 1;
""", conn)

# Most Frequently traded punk
pd.read_sql_query("""
SELECT PunkID, COUNT(*) AS TradeCount
FROM PunkTrades
GROUP BY PunkID
ORDER BY TradeCount DESC
LIMIT 1;
""", conn)

# Top 5 traders + average net USD position
pd.read_sql_query("""
WITH trader_flows AS (
    SELECT TFrom AS Trader, TAmt AS USDFlow
    FROM PunkTrades
    WHERE TAmt > 0 AND TFrom IS NOT NULL AND TFrom != ''

    UNION ALL

    SELECT TTo AS Trader, -TAmt AS USDFlow
    FROM PunkTrades
    WHERE TAmt > 0 AND TTo IS NOT NULL AND TTo != ''
```

```
)
SELECT
    Trader,
    COUNT(*) AS NumTrades,
    AVG(USDFlow) AS AvgNetPositionUSD
FROM trader_flows
GROUP BY Trader
ORDER BY NumTrades DESC
LIMIT 5;
""", conn)
```

Task C: Fred Data Download

```
# Load FRED CSV
eth_df = pd.read_csv("CBETHUSD.csv")

eth_df.rename(columns={
    "observation_date": "DATE",
    "CBETHUSD": "CloseUSD"
}, inplace=True)

# Convert date
eth_df["DATE"] = pd.to_datetime(eth_df["DATE"],  format="%d-%m-%Y")

# Convert to YYYYMMDD integer
eth_df["TDate"] = eth_df["DATE"].dt.strftime("%Y%m%d").astype(int)

# Keep required columns
eth_df = eth_df[["TDate", "CloseUSD"]].dropna()

eth_df.head()

conn.execute("DROP TABLE IF EXISTS EthPrices;")

conn.execute("""
CREATE TABLE EthPrices (
    TDate INTEGER PRIMARY KEY,
    CloseUSD REAL
);
""")

eth_df.to_sql("EthPrices", conn, if_exists="append", index=False)

conn.commit()

pd.read_sql_query(
    "SELECT name FROM sqlite_master WHERE type='table';",
    conn
)

import pandas as pd
import matplotlib.pyplot as plt
import sqlite3

# Connect to DB
conn = sqlite3.connect("punkDB.db")
```

```
# Load ETH data from SQLite
eth = pd.read_sql_query(
    "SELECT TDate, CloseUSD FROM EthPrices ORDER BY TDate;",
    conn
)

# Convert integer date to datetime
eth["Date"] = pd.to_datetime(eth["TDate"].astype(str), format="%Y%m%d")

# Plot
plt.figure(figsize=(12,6))
plt.plot(eth["Date"], eth["CloseUSD"], color="purple", linewidth = 1.5)
plt.title("ETH-USD Daily Price (Feb 2016 - Present)", fontsize=14)
plt.xlabel("Date")
plt.ylabel("Price (USD)")
plt.grid(True)
plt.tight_layout()
plt.show()
plt.savefig('ETH-USD Daily Price (Feb 2016 - Present).png', dpi=300,
bbox_inches='tight')
```

Task D : Hedonic Regression

```
# Count sales per punk
sales_count = pd.read_sql_query("""
SELECT PunkID, COUNT(*) as NumSales
FROM PunkTrades
WHERE TType='Sold' AND TAmt > 0
GROUP BY PunkID
HAVING COUNT(*) >= 2;
""", conn)

eligible_punks = tuple(sales_count["PunkID"].tolist())

len(eligible_punks)

# TRUE Case-Shiller Repeat-Sales Index

import numpy as np
import statsmodels.api as sm

# Load Sold transactions
sold = pd.read_sql_query("""
SELECT TDate, PunkID, TAmt
FROM PunkTrades
WHERE TType='Sold'
AND TAmt > 0
ORDER BY PunkID, TDate;
""", conn)

sold["Date"] = pd.to_datetime(sold["TDate"].astype(str))
sold["LogPrice"] = np.log(sold["TAmt"])
sold = sold.sort_values(["PunkID","Date"])
```

```
# Build repeat-sales pairs
repeat_pairs = []

for punk, group in sold.groupby("PunkID"):
    group = group.sort_values("Date")
    if len(group) < 2:
        continue
    for i in range(1, len(group)):
        prev = group.iloc[i-1]
        curr = group.iloc[i]
        repeat_pairs.append({
            "Date1": prev["Date"],
            "Date2": curr["Date"],
            "LogReturn": curr["LogPrice"] - prev["LogPrice"]
        })

rs = pd.DataFrame(repeat_pairs)

# Convert to monthly periods
rs["Period1"] = rs["Date1"].dt.to_period("M")
rs["Period2"] = rs["Date2"].dt.to_period("M")

# Get all months
all_periods = sorted(
    set(rs["Period1"]).union(set(rs["Period2"]))
)

period_index = {p: i for i,p in enumerate(all_periods)}

# Build X matrix
T = len(all_periods)
N = len(rs)

X = np.zeros((N, T))

for i, row in rs.iterrows():
    X[i, period_index[row["Period2"]]] = 1
    X[i, period_index[row["Period1"]]] = -1

y = rs["LogReturn"].values

# Drop first column to avoid dummy trap
X = X[:,1:]

# Run regression
model_csm = sm.OLS(y, X).fit()

# Build full dummy vector
betas = np.zeros(T)
betas[1:] = model_csm.params

# Direct log index = betas
index_log = betas

# Normalize first period to 0
index_log = index_log - index_log[0]
```

```
# Convert to level
index_level = np.exp(index_log)

# Base = 100
index_level = 100 * index_level / index_level[0]

cs_index = pd.DataFrame({
    "Period": all_periods,
    "IndexLevel": index_level
})

print("Case-Shiller Index Built Successfully")
rs["Period1"].value_counts()

cs_index["Date"] = cs_index["Period"].dt.to_timestamp()

plt.figure(figsize=(12,6))
plt.plot(cs_index["Date"], cs_index["IndexLevel"], color="purple")
plt.title("CryptoPunks Case-Shiller Repeat-Sales Index")
plt.xlabel("Date")
plt.ylabel("Index Level")
plt.grid(True)
plt.tight_layout()
plt.show()

# Rarity Measure
import math
punks = pd.read_sql_query(
    "SELECT PunkID, PunkType, AttributeVec FROM Punks;",
    conn
)

attr_len = len(punks["AttributeVec"].iloc[0])
counts = [0] * attr_len

for vec in punks["AttributeVec"]:
    for i, ch in enumerate(vec):
        if ch == "1":
            counts[i] += 1

def rarity_score(vec):
    present = [counts[i] for i, ch in enumerate(vec) if ch == "1"]
    if len(present) == 0:
        return None
    return -math.log(min(present))

punks["RarityScore"] = punks["AttributeVec"].apply(rarity_score)
print(punks)

import numpy as np

total_punks = len(punks)

# Sum-Based Rarity Score
def rarity_sum(vec):
    score = 0
    for i, ch in enumerate(vec):
```

```
        if ch == "1":
            prob = counts[i] / total_punks
            score += -np.log(prob)
    return score

punks["RaritySumScore"] = punks["AttributeVec"].apply(rarity_sum)

# Number of Traits (Collectibility proxy)
punks["NumTraits"] = punks["AttributeVec"].apply(lambda v: v.count("1"))

trades = pd.read_sql_query("""
SELECT TDate, PunkID, TAmt, TType
FROM PunkTrades
WHERE TType IN ('Sold', 'Offered')
AND TAmt > 0;
""", conn)

eth = pd.read_sql_query("""
SELECT TDate, CloseUSD FROM EthPrices;
""", conn)

df = trades.merge(
    punks[["PunkID", "PunkType", "RarityScore", "RaritySumScore",
"NumTraits"]],
    on="PunkID",
    how="left"
)

df = df.merge(eth, on="TDate", how="left")

# Add Year column
df["Year"] = pd.to_datetime(df["TDate"].astype(str)).dt.year

# Prepare Case-Shiller yearly index
cs_index["Year"] = cs_index["Period"].dt.year

# Merge yearly Case-Shiller index
df = df.merge(
    cs_index[["Year", "IndexLevel"]],
    on="Year",
    how="left"
)

df["LogPriceUSD"] = np.log(df["TAmt"])
df["LogETHUSD"] = np.log(df["CloseUSD"])
df["LogIndex"] = np.log(df["IndexLevel"])

df = df.dropna().copy()

type_dummies = pd.get_dummies(
    df["PunkType"],
    drop_first=True
)

df = pd.concat([df, type_dummies], axis=1)

# Split 2/3 by date
```

```
unique_dates = sorted(df["TDate"].unique())

cut = int(len(unique_dates) * (2/3))

is_dates = unique_dates[:cut]
os_dates = unique_dates[cut:]

is_df = df[df["TDate"].isin(is_dates)].copy()
os_df = df[df["TDate"].isin(os_dates)].copy()


df.head()
# Hedonic Regression
# Convert boolean dummies to int
# Define regression columns FIRST
X_cols = ["RaritySumScore", "NumTraits", "LogIndex", "LogETHUSD"] +
list(type_dummies.columns)

# Convert boolean dummies to int
is_df[type_dummies.columns] = is_df[type_dummies.columns].astype(int)

# Ensure numeric
is_df[X_cols] = is_df[X_cols].apply(pd.to_numeric, errors='coerce')
is_df["LogPriceUSD"] = pd.to_numeric(is_df["LogPriceUSD"], errors='coerce')

# Drop missing
is_df = is_df.dropna(subset=X_cols + ["LogPriceUSD"])

# Build regression
X = sm.add_constant(is_df[X_cols])
y = is_df["LogPriceUSD"]

hedonic_model = sm.OLS(y, X).fit()

print(hedonic_model.summary())
```

Task 4 : Out_of_sample Forecast

```
# Part 4
# Convert TDate to datetime
os_df["Date"] = pd.to_datetime(os_df["TDate"].astype(str))

# Identify first date in OS
start_os = os_df["Date"].min()

# Define 6 month window
end_window = start_os + pd.DateOffset(months=6)

trade_window = os_df[
    (os_df["Date"] >= start_os) &
    (os_df["Date"] <= end_window)
].copy()

len(trade_window)

# Ensure numeric types for trading window
```

```
trade_window[X_cols] = trade_window[X_cols].apply(pd.to_numeric,
errors='coerce')
trade_window[type_dummies.columns] =
trade_window[type_dummies.columns].astype(int)
trade_window = trade_window.dropna(subset=X_cols)

X_trade = trade_window[X_cols].copy()
X_trade = sm.add_constant(X_trade, has_constant='add')

trade_window["PredictedLogPrice"] = hedonic_model.predict(X_trade)
trade_window["PredictedPrice"] = np.exp(trade_window["PredictedLogPrice"])
print(trade_window["PredictedLogPrice"])

trade_window["Signal"] = 0

trade_window.loc[
    trade_window["TAmt"] < trade_window["PredictedPrice"],
    "Signal"
] = 1   # BUY

trade_window.loc[
    trade_window["TAmt"] > trade_window["PredictedPrice"],
    "Signal"
] = -1  # SHORT

trade_window["SignalLabel"] = trade_window["Signal"].map({
    1: "BUY (Undervalued)",
    -1: "SHORT (Overvalued)",
    0: "No Trade"
})

trade_window = trade_window.sort_values("Date")

trade_window = trade_window.drop_duplicates(
    subset=["PunkID", "Date", "TAmt", "TType"]
)

returns = []
open_positions = set()

for idx, row in trade_window.iterrows():

    punk = row["PunkID"]

    # Skip if already holding position in this punk
    if punk in open_positions:
        continue

    if row["Signal"] == 0:
        continue

    entry_price = row["TAmt"]
    entry_date = row["Date"]
    signal = row["Signal"]

    # Find next SOLD trade after entry
    future_sales = os_df[
```

```
        (os_df["PunkID"] == punk) &
        (pd.to_datetime(os_df["TDate"].astype(str)) > entry_date) &
        (os_df["TType"] == "Sold")
    ].sort_values("TDate")

    if len(future_sales) == 0:
        continue

    exit_price = future_sales.iloc[0]["TAmt"]

    if signal == 1:
        ret = (exit_price - entry_price) / entry_price
    else:
        ret = (entry_price - exit_price) / entry_price

    returns.append(ret)

    # Mark as closed immediately (since we exit at next sale)
avg_return = np.mean(returns)
print("Closed trades:", len(returns))
print("Average return:", avg_return)

trade_window["MispricingPct"] = (trade_window["PredictedPrice"] -
trade_window["TAmt"]
                                )/ trade_window['PredictedPrice']
trade_window["MispricingPct"] = trade_window["MispricingPct"].round(3)
trade_window["SignalStrength"] = abs(trade_window["MispricingPct"])
trade_window["SignalLabel"] = trade_window["Signal"].map({
    1: "BUY",
    -1: "SHORT",
    0: "No Trade"})
trade_window[[
    "PunkID",
    "Date",
    "TAmt",
    "PredictedPrice",
    "MispricingPct",
    "SignalLabel"
]].head(20)
print("Min mispricing:", trade_window["MispricingPct"].min())
print("Max mispricing:", trade_window["MispricingPct"].max())
print("Number of BUY:", (trade_window["Signal"] == 1).sum())
print("Number of SHORT:", (trade_window["Signal"] == -1).sum())
```